DIAGNOSTIC & REPAIR REPORT · 28 AUG 2026

Why it never bought, and never *sold*

Five parallel audits over ~40,000 lines, then a 1,000-scenario sweep to prove the fixes. The system wasn't under-tuned — two independent gates made buying arithmetically impossible, and four stacked faults made selling impossible. All are now fixed and verified end-to-end, dashboard included. One thing I could not fix is the 10%-a-day target.

The one thing to read first

The 10% daily target

NOT ACHIEVABLE — THIS IS ARITHMETIC, NOT PESSIMISM

You asked for 10% per day, compounding, with the profit rolled back in. I can't build that, and neither can anyone else. Compounding ₹12,000 at 10% a day for one trading year gives a number larger than the entire Indian equity market.

I've built everything else you asked for and made the system genuinely capable of trading. But I'd be lying to you if I set the target engine to 10%/day and let it run.

START	1 WEEK	1 MONTH	3 MONTHS
₹12,000	₹19,326	₹80,730	₹36.5 lakh
6 MONTHS	**1 YEAR**		
₹179 crore	₹2.7 × 10^{14}		

There is a second, harder wall underneath the maths. Every trade pays costs. With the system's current delivery (CNC) product type and three round trips a day, **1.86% of deployed capital is consumed by charges daily before a single rupee of profit**. To net 10% you'd need to gross **11.9% every single day** — more than most NSE scrips can move before hitting their circuit limit.

WHAT I SET INSTEAD

I've left your configured daily target untouched at ₹1,200 so nothing silently changes under you, but the honest framing is this: a well-built intraday system that reliably nets **0.3%–0.8% a day** after costs is already performing well. On ₹12,000 that is ₹36–₹96 a day, and it compounds. The engineering below is what makes that possible; no engineering makes 10% possible.

Two structural notes on the ₹12,000 itself: at that size a single ₹4,000 share is a third of your book, so position sizing is coarse, and fixed per-trade charges hurt disproportionately — the flat ₹15.93 DP charge alone is 40 bps on a ₹4,000 delivery ticket.

Root cause · buying

The funnel died at the same step, every time

Your database had already recorded the evidence. Across every logged cycle the pipeline collapsed at exactly one place.

Universe	5,000
Scanned	4,966
Researched	400
Ranked	200
Candidates	14
Consensus passed	0–1
Orders placed	0

Of the last 50 recorded decisions, **48 were HOLD and 2 were BUY** — and neither BUY reached an order. Digging into the agent votes showed why: **6 of the 13 agents emitted the identical vote every single time**, regardless of input. They carried voting weight but zero information.

The decisive one was `agent11_price_action` — the *primary* trigger in what the code itself calls a "candle-first" engine. It logged reasoning like *"PA-Structure confirms bullish structure (hh + hl) (Score: 85). EMA9/21 is bullish"* — and then returned **HOLD at exactly 0.50 confidence**, 50 times out of 50.

Three vetoes that all fire hardest when a trend is strongest

I reproduced this in isolation. Fed a clean +34% uptrend with bullish EMAs, the agent returned HOLD — because all three of its safety rules tripped simultaneously:

FAILSAFE RULE	REQUIRED	ACTUAL ON A STRONG UPTREND	RESULT
Volume expansion	RVOL ≥ 1.2	1.0 max, structurally	always veto
Structure score	≥ 60	50 (pinned)	always veto
Risk : reward	≥ 1.5	0.02	always veto

Each had its own cause. RVOL compared the *currently-forming* intraday bar (a fraction of a full bar's volume) against a full-bar average, so the ratio could never reach 1.2. Structure detection used strict swing pivots, and a clean advance contains *no* qualifying pivots — so it always fell through to the neutral 50. And risk:reward was computed as *distance-to-resistance ÷ distance-to-support*, which demands price sit near the *bottom* of its range to buy — the exact opposite of a breakout.

Together they formed a contradictory specification: *confirm a strong uptrend, but only buy near the range low, on a volume spike in a bar that hasn't finished forming.* Those conditions cannot co-occur. The agent was mathematically incapable of ever emitting BUY.

Root cause · selling

Four faults stacked on the same safety net

The end-of-day square-off is the one path that must never fail. It was broken in four independent ways at once — and the fourth was actively dangerous.

- **It was unreachable.** `tick()` returns early for any state that isn't SCANNING or TRADING. The square-off was guarded by `state === 'MARKET_CLOSING'` — one of the states that early return rejects. It had never executed, not once.
- **It would have thrown anyway.** `riskEngine` was never imported at module scope, so the call site raised a `ReferenceError` caught by a bare log-only handler.
- **It called a method that doesn't exist.** The function invoked `broker.placeOrder(...)`. The broker exports `executeOrder`. Every position threw `TypeError`, silently swallowed.
- **Then it falsified the ledger.** After every sell failed, it unconditionally ran `holding_stocks = []` and saved — and sent Telegram "Liquidating all positions". The book read flat while every real position stayed open at the broker, with nothing left to monitor them.

WHY THIS MATTERS MORE THAN THE BUY BUG

A system that doesn't buy loses you nothing. A system that believes it has closed positions it still holds — with no stop-loss monitoring and no reconciliation — is how an account gets damaged overnight. This was the single most dangerous defect found.

The changes

Before and after

Every item below was verified against the actual code before being changed, and is covered by the regression suite.

Execution & exits

Price-action agent could never signal BUY — CRITICAL — priceActionStructureAgent.js

BEFORE

Three contradictory hard vetoes. **0 BUY signals in 50 recorded decisions.**

AFTER

RVOL measured on *closed* bars; structure falls back to a rolling-window read; R:R computed from an ATR stop and a measured move. Vetoes became graded confidence penalties. **67% BUY capture on uptrends, 0.7% on downtrends.**

One pattern detector could override the whole composite score — HIGH — priceActionStructureAgent.js

BEFORE

`adv.score > 70` was an unconditional OR. Measured: **11 of 12 false BUYs inside confirmed downtrends** came from this bypass alone.

AFTER

An advanced pattern may trigger only when market structure isn't actively opposing it. A bull flag in a downtrend is usually a continuation pause, not a reversal.

Decision engine grade threshold was unreachable

CRITICAL · adaptiveDecisionEngine.js

BEFORE

Execution required grade A (composite ≥ 70). Every component is anchored at a neutral 50 and the candle term is discounted, so real setups top out near 75. Measured across 1,000 generated scenarios: **0 executed**. Feeding 700 real agent outputs through it, only **~5% of valid BUY setups** could ever pass.

AFTER

Bands measured against the actual score distribution (BUY setups: p50 = 61, p90 = 65, max = 75) and set to A+ ≥ 68, A ≥ 62. Execution rate went from **0.1% to 20%**. Selectivity now lives in the top-5 TQS ranking, not in an unreachable cutoff.

End-of-day square-off never ran

CRITICAL · tradingBot.js · riskEngine.js

BEFORE

Unreachable code, missing import, non-existent broker method — then it blanked `holding_stocks` regardless of whether anything sold.

AFTER

Moved above the state gate and onto `AUTO_SQUAREOFF_TIME` (15:15 IST, ahead of the broker's own forced square-off). Sells via `executeOrder` with 3 retries, pauses new entries first, books realised P&L, and **leaves unsold positions on the book** with a CRITICAL alert.

Stop-loss monitoring stopped 5 minutes before the close

HIGH · tradingBot.js

BEFORE

Exits ran only in SCANNING/TRADING, which end at 15:25 — leaving the day's most volatile window unmonitored.

AFTER

`processRealExits()` now runs through the closing window whenever the session is open.

"/stop" silently disabled all risk management

CRITICAL · tradingBot.js

BEFORE

`entriesPaused` returned from the top of `tick()`, killing stop-losses, targets, the FSM clock and end-of-day processing. Its own auto-reset line sat *below* that return, so it could never clear itself.

AFTER

Gates the entry path only. Exits, risk checks and the daily reset keep running.

Position-replacement exit read a field that doesn't exist

HIGH · tradingBot.js

BEFORE

`pos.entry_price` → `undefined` → P&L was `NaN`, and `NaN <= 0` is false, so the branch never fired.

AFTER

Reads `pos.avgPrice`, the field the broker actually writes.

Live-money safety

Orders were never confirmed; trades booked at a Yahoo quote

CRITICAL · new · kiteOrders.js

BEFORE

Kite's `status:"success"` means *accepted into the queue*, not filled. The response was discarded — no order id, no status poll — and the trade was recorded at the pre-trade quote. Every entry price, exit price and P&L figure was fiction; slippage was unobservable.

AFTER

New module places the order once, tags it, then polls to a terminal state and returns the real `filled_quantity` and `average_price`. Nothing reaches the ledger unless the exchange confirms COMPLETE; anything else raises a CRITICAL alert and refuses.

A network hiccup could place the same market order three times

CRITICAL · broker.js

BEFORE

The order POST was wrapped in `withResilience(..., 3 retries)`. `fetch` rejects on network errors — exactly the ambiguous case where the order may already have routed. The 4xx guard was dead code (it tested `err.response`, an axios idiom fetch never sets).

AFTER

Order placement is never retried. Reads keep their retry wrapper.

Orders could be booked with none ever sent to an exchange

CRITICAL · broker.js

BEFORE

The broker dispatch chain had no final `else`. During async startup, or permanently after a failed login, an order fell straight through to the ledger and was recorded as filled.

AFTER

Explicit `liveBrokerState` distinguishes deliberate paper trading from a missing or failed session. The second case throws rather than recording a phantom fill.

Expired Kite token would fail every order, all day, silently

CRITICAL · broker.js · kiteOrders.js

BEFORE

Session verified once at boot and never again. Kite tokens expire daily ~06:00 IST. From day two under PM2, entries *and exits* would fail into a log-and-continue handler with open positions and no alert.

AFTER

`TokenException` and HTTP 403 are detected, the session is cleared, LIVE routing is halted and a Telegram alert fires. `verifyToken()` is available for periodic re-checks.

Random-walk candles were labelled as live market data

CRITICAL · marketData.js · predictor.js

BEFORE

When Yahoo was unreachable, a synthetic random walk with a built-in +0.1% drift was returned tagged `source:'LIVE'`. The predictor, SMC agent and stop/target engine could generate BUY signals from pure noise.

AFTER

Tagged `SYNTHETIC` / `DEGRADED_LTP_ONLY`, and the predictor refuses to produce a signal on either. Not trading beats trading on fabricated data.

OHLCV arrays were filtered independently, desynchronising bars

HIGH · marketData.js

BEFORE

Yahoo emits `null` per-field on illiquid bars. Filtering each array separately shifted them relative to one another, so `candle[i]` mixed one bar's open with another's high. Every structural and ATR calculation read fabricated candles.

AFTER

Filtered bar-wise in a single pass — a bar is dropped only if it is itself incomplete.

A failed stop-loss was re-blocked as a "duplicate" for four ticks

HIGH · broker.js

BEFORE

The 2-second duplicate window covered SELLs too, while the tick interval is 500 ms — so a transiently failed exit was rejected on each of the next four attempts, precisely when the position needed closing.

AFTER

Duplicate suppression applies to entries only.

A NaN quantity passed every guard and corrupted the ledger

HIGH broker.js

BEFORE

`NaN <= 0` is false, so it passed the quantity check; `balance < NaN` is also false, so it passed the balance check. Kite would receive `quantity:"NaN"` and the ledger balance became permanently `NaN`.

AFTER

Requires a finite positive integer. Verified against NaN, 0, negative, fractional and Infinity.

Lifetime P&L measured against a hardcoded ₹12,000

HIGH broker.js

BEFORE

Connecting a ₹50,000 Zerodha account would instantly report **+₹38,000 profit**; a ₹5,000 account would sit below the lifetime floor and **halt on boot**. That figure drives the halt logic.

AFTER

Resolved from a persisted `capital_baseline`, captured once, with the constant only as a fallback.

The two most safety-critical alerts were silently discarded

HIGH db.js

BEFORE

Order-failure and price-source-mismatch alerts called `logAlert('CRITICAL', msg)` positionally, but the function takes an object. `type` and `message` came out `undefined`, violating NOT NULL — the alerts vanished in both storage modes.

AFTER

Accepts both call styles. Both forms covered by tests.

Timing & the Indian market session

Session logic ran on UTC, shifting every rule by 5h30m

HIGH predictor.js · agentResearch.js

BEFORE

`db.getSystemTime` *does not exist*, so a ternary always fell through to `new Date().getHours()` — UTC in production. At 15:15 IST the classifier reported "OPENING_AUCTION", 15 minutes before the close.

AFTER

All four sites route through `lifecycleFSM.getSystemTime()`, the one correct IST source (it uses `Intl` with `Asia/Kolkata`).

> **ALREADY CORRECT — LEFT ALONE**
>
> The core session model was sound and I didn't touch it: `getSystemTime()` derives IST properly via `Intl.DateTimeFormat`, an NSE holiday calendar exists and is wired into the session gate, and the bot already refuses to scan on weekends and holidays. The 09:15–15:30 windows were right. The bugs were all in code that *bypassed* that logic.

Cost model

The system used a flat 0.05% per leg everywhere. That's roughly right for intraday and badly wrong for delivery — which is what the live path actually places, because `tradingBot` passes `'CNC'`. The new `brokerCharges.js` models the full statutory stack including the flat per-scrip DP charge, which a percentage model structurally cannot express.

₹4,000 ROUND TRIP	OLD FLAT MODEL	REAL COST	UNDERSTATED BY
Intraday (MIS)	₹4.00 · 10.0 bps	₹4.24 · 10.6 bps	≈ accurate
Delivery (CNC) — what it actually uses	₹4.00 · 10.0 bps	₹24.83 · 62.1 bps	6.2×

Every backtest and every learning-engine weight update was systematically optimistic by that margin.

Dashboard

- **Position cards showed a stop the engine never uses.** The code read `h.stopLoss`; the field is `stopLossPrice`. Every card rendered a synthetic `avgPrice × 0.97` while the engine enforced something else entirely — the card was misreporting your actual risk.
- **The Stop button always sent START.** `isBotRunning` was declared and never assigned, so the stop path was unreachable even though the label flipped to "Stop Bot".
- **The bot status dot was red while running.** The healthy branch set a bare `status-dot` class, which CSS paints red. A running bot looked offline.
- **The MARKET CLOSED banner could never appear.** It tested a key that doesn't exist in the payload.
- **The rejection modal was dead twice over.** It read `window._lastStatusData`, which is never assigned, from a field that is always empty because the writer calls a non-existent method. It now reads the real rejection records, which had been broadcast in every frame all along.

- **New: Live Risk & Execution Blockers panel.** Real rupee capital-at-risk from actual stops, positions with *no* stop flagged in red, open exposure, daily loss budget consumed (the dashboard showed the limit but never the usage), entry-gate state, and a ranked list of which gate blocked each candidate — with near-misses highlighted. All computed from data already on the wire.
- **The dashboard had no chart at all.** ~540 lines of charting code existed but `initChart()` was never called and no container existed to mount into — while `index.html` still downloaded a 200KB charting library on every page load. There is now a Price Action panel with 5-minute candles, EMA 9/21, VWAP and an RSI sub-chart, loading ~280 real bars, with retry-and-report instead of failing silently.
- **The Ledger tab was eight hardcoded numbers.** Total trades, win rate, profit factor, Sharpe, drawdown, net P&L, average winner and loser were static markup that no JS ever assigned — permanently reading 0 / 0.0% / 1.00 / 0.00 as though measured. Now driven from `paperTradingStats`, which was in every broadcast frame already.
- **The Scanner tab read zero while the scanner was running.** The backend called `marketScanner.getScannerStats()`; the method is `getMetrics()`, with different key names. The "if the method exists" ternary therefore always took its zeroed fallback. It now reports the real figures — a lifetime count of **1,903,192 symbols scanned** where it previously showed 0.
- **Unknown is no longer displayed as zero.** The backend deliberately sends `null` for "no trades yet, win rate unknown", but the frontend rendered `Number(null).toFixed(1)` — a confident **0.0%**. A measured zero and an unknown must not look identical on a trading screen; unknowns now render as an em-dash.
- **Layout overflowed the viewport.** Measured with an automated probe at 375 / 768 / 1440 / 1920 px. The main grid pushed its third column 81px off-screen because the charting canvas could not shrink, and the header ran to 744px against a 375px phone. Fixed with `minmax(0, 1fr)` tracks, a ResizeObserver that keeps the chart in step with its container, and scroll containment on the timeline. All four widths now report zero overflowing elements.
- **Output is now escaped.** Server text went into `innerHTML` unescaped in ~9 places. Telegram alert bodies are HTML, so tags were rendering as live markup; LLM reasoning and raw upstream error strings flowed to the same sinks.

Evidence

How this was verified

I wrote a reproduction harness before changing anything, then a regression suite that runs without network access.

1,000
scenarios, zero invariant violations

53×
uptrend vs downtrend separation

21 / 21
regression tests passing

104
malformed orders, none accepted

Behaviour across 1,000 generated scenarios

Eight market regimes, 30–120 bars each, prices spanning ₹12 to ₹24,500, 8% of series with no volume at all. Every case is driven through the real signal, decision, sizing and cost code. A clean monotonic gradient across regimes is the signature of a genuinely discriminating engine — before the fix this column read 0% everywhere.

REGIME	N	BUY	SELL	HOLD
Strong uptrend	129	90.7%	0.0%	9.3%
Weak uptrend	126	63.5%	0.0%	36.5%
Melt-up (very noisy)	126	46.0%	0.0%	54.0%
Sideways	132	39.4%	0.0%	60.6%
Volatile	128	28.1%	3.9%	68.0%
Weak downtrend	125	19.2%	0.0%	80.8%
Strong downtrend	104	1.9%	0.0%	98.1%
Crash	130	0.8%	12.3%	86.9%

THREE DEFECTS THE 1,000-CASE SWEEP FOUND THAT SMALLER TESTS MISSED

The decision engine still passed nothing. Fixing the signal agent was only half the funnel — the grade threshold was independently unreachable. 0 of 1,000 executed until it was calibrated.

Directional signals below 0.5 confidence. Quality penalties could drag a SELL to 0.45 — "bearish, but less than neutrally sure", which is incoherent for any downstream weighting. Such calls are now downgraded to HOLD.

The test harness itself was biased. A plain LCG consumed at a variable stride skewed regime sampling roughly 10:1, which would have made every statistic above untrustworthy. Replaced with mulberry32; sampling is now even (104–132 per regime).

The signal threshold wasn't guessed. I swept it against 600 synthetic-but-realistic price series across four regimes and picked the value with the best separation — 67% capture on uptrends against 0.7% on downtrends. Run the suite yourself:

COMMAND	WHAT IT DOES
`node backend/diagnostics/validate_1000.js`	1,000-scenario behavioural sweep with population assertions
`node backend/diagnostics/verify_fixes.js`	21-test regression suite, no network needed
`node backend/diagnostics/repro_signal_veto.js`	Reproduces the original never-buys defect

Honest limits

What is still open

These are real, they were found and verified, and I did not fix them. Read this section before connecting live credentials.

- **No broker reconciliation loop.** I built the pieces — `fetchNetPositions()` and `fetchHoldings()` in `kiteOrders.js` — but nothing periodically diffs the broker's real positions against the local ledger. This is the single most important remaining item: if the two ever diverge, a SELL for shares you don't hold becomes a short delivery, which carries an auction penalty of up to 20% of trade value.
- **Lost updates on `db.json`.** Two concurrent orders can each read the balance, then each write — one silently overwrites the other, fabricating cash or dropping a position. There is no lock; a declared `isWriting` flag was never implemented.

- **Postgres schema does not match the code.** Seven tables are defined incompatibly between `schema.sql` and the runtime DDL, and `CREATE TABLE IF NOT EXISTS` won't fix an existing wrong-shaped table. Measurable effect: **230 records across five tables in your live `db.json` are marked `synced: false` and have never reached Neon.**
- **Five agents still emit near-constant votes.** I fixed agent 11 because it was the decisive blocker. Agents 1, 5, 7, 9 and 10 still return the same signal almost every time — they dilute the consensus without adding information. Worth rebuilding or removing.
- **Product type is overloaded.** `strategy` doubles as both a ledger key and the Kite product type, so an intraday bot places delivery (CNC) orders — which is why costs are 6× the model and nothing is auto-squared by the exchange.
- **A restart mid-session resets the day's P&L baseline.** This also re-zeroes the 3% daily loss guard and can clear a HALTED status. Your logs show four restarts across two days, so it isn't hypothetical.

> ON THE CLEANUP YOU ASKED FOR
>
> I moved `market_scanner.js` out of `scratch/` into `backend/` where it belongs — it was a production module living in a scratch folder — and removed three dead imports. I did *not* mass-delete the ~190 scratch files: 23 of them are untracked, and `historicalReplayEngine.js` and `shared/domains.js` are files you created and haven't committed. Deleting untracked work on my own judgement isn't a call I should make. Commit what you want to keep and I'll clear the rest in one pass.
>
> Worth knowing: `scratch/` contains 12 Python scripts that rewrite backend source files in place using June-era string replacements, plus 10 scripts that wipe the database with no confirmation. Those are worth deleting whether or not you clean up anything else.

Before live credentials

The order I'd do this in

- **Run it in paper mode for two weeks first.** It has now traded zero times in its life. Its win rate, its slippage, its actual cost drag are all unknown. Nothing about live capital should be decided before there's real data.
- **Wire the reconciliation loop.** Startup, then every 60 seconds, and again at 15:10. On any mismatch: halt entries, alert, never sell an unconfirmed quantity.
- **Switch the intraday path to MIS.** Cuts round-trip cost from ~62 bps to ~11 bps and restores exchange-side auto square-off as a backstop.
- **Fix the ledger write race and the Postgres schema.** Until then the learning engine is training on partial data.

- **Start with capital you can afford to lose entirely.** Not because the code is bad — it's meaningfully better than it was — but because it has no live track record at all.

Findings produced by five parallel subsystem audits across ~40,000 lines, then re-verified by a 1,000-scenario behavioural sweep that itself surfaced three further defects. Every claim was checked against the source before any change was made. 19 files modified, 5 added, 1 relocated. All backend, shared and frontend files parse cleanly; 21 of 21 regression tests pass; 1,000 of 1,000 scenarios pass with no invariant violations; the dashboard was verified running against the live server.

Nothing here is investment advice. I'm not a licensed financial adviser — the return figures above are arithmetic about the code and its costs, not a recommendation about your money.